



The Ultimate AI

LANGGRAPH CHEATSHEET

Your pocket guide to Agentic AI.



Chandra

Senior GenAI Engineer



Import Necessary Modules

```
from typing import TypedDict, Dict, Any, Callable
from langchain_core.runnables import Runnable
from langgraph.graph import StateGraph, END
```

Define the State

- Use a TypedDict to define the structure of your state.
- This is the shared data passed between nodes.

```
class MyGraphState(TypedDict):
    input_data: str
    intermediate_result: str
    final_output: str
```

Create Nodes

- Nodes are functions or Runnables that process the state.
- They take the current state as input and return updates to the state.



```
def node_a(state: MyGraphState) → Dict[str, Any]:  
    new_result = f"Processed: {state['input_data']}"  
    return {"intermediate_result": new_result}
```

Using LangChain Runnables as Nodes:



```
from langchain_core.prompts import ChatPromptTemplate  
from langchain_openai import ChatOpenAI  
  
prompt = ChatPromptTemplate.from_template("Tell me a joke about {topic}")  
model = ChatOpenAI()  
runnable_node = prompt | model # Combine prompt and model as a Runnable
```

Create the Graph

- Instantiate a StateGraph with your state definition.



```
builder = StateGraph(MyGraphState)
```

Add Nodes to the Graph



```
builder.add_node("node_a", node_a)  
builder.add_node("runnable_node", runnable_node)
```

Define Edges

- Direct edges connect nodes in sequence.



```
builder.add_edge("node_a", "runnable_node")
```

Conditional Edges

- Use conditional edges for branching logic.



```
def should_continue(state: MyGraphState):  
    if len(state["intermediate_result"]) > 10:  
        return "runnable_node" # Route to runnable_node  
    else:  
        return END # End the graph  
  
builder.add_conditional_edges(  
    "node_a",  
    should_continue,  
    {"runnable_node": "runnable_node", END: END}  
)
```

Set the Entry Point

- Define where execution starts in the graph.



```
builder.set_entry_point("node_a")
```

Compile the Graph

- Compile the graph into a RunnableGraph.



```
graph = builder.compile()
```

Invoke the Graph

- Pass an initial state to execute the graph.



```
inputs = {"input_data": "Short input"}  
result = graph.invoke(inputs)  
print(result)
```

Parallel Execution

- Use Send() to dispatch tasks to multiple nodes in parallel and Gather() to collect results.

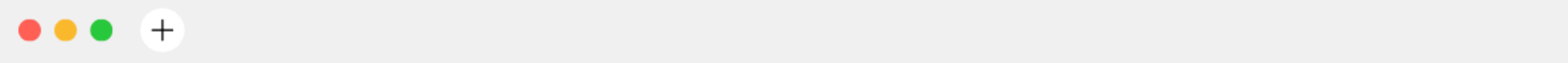


```
from langgraph.graph import Send
```

```
# Example parallel execution (not fully shown here)  
# builder.add_node("parallel_task", Send(...))
```


Checkpoints

- Save and restore the state of the graph at specific points (useful for long-running workflows).



```
from langgraph.graph import CheckpointAt

checkpoint_config = CheckpointAt(
    nodes=["node_a", "runnable_node"], # Nodes to checkpoint
    config={"save": True, "load": True}
)
```

Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save